

HOST sending function:

- takes application message
- breaks into smaller chunks, *packets*, of length L bits
- transmits packet into access network at *transmission rate* R (link bandwidth)

$$\frac{\text{packet transmission}}{\text{delay}} = \frac{\text{time needed to transmit } L\text{-bit packet into link}}{\frac{L \text{ (bits)}}{R \text{ (bits/sec)}}}$$

Packet-switching: *store and forward*: entire packet must arrive at router before it can be transmitted on next link $1k=10^3$, $1M=10^6$, $1G=10^9$, $1T=10^{12}$

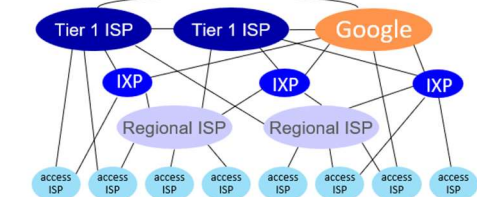
Packet queuing and loss: if arrival rate (in bps) to link exceeds transmission rate (bps) of link will queue, waiting to be transmitted on output link can be dropped (lost) if memory (buffer) in router fills up

Circuit switching: end-end resources allocated to, reserved for “call” between source and destination

- Frequency Division Multiplexing (FDM): optical, electromagnetic frequencies divided into (narrow) frequency bands. Each call allocated its own band, can transmit at max rate of that narrow band
- Time Division Multiplexing (TDM): time divided into slots. Each call allocated periodic slot(s), can transmit at maximum rate of (wider) frequency band (only) during its time slot(s)

Internet structure: a “network of networks”

- hosts connect to Internet via access Internet Service Providers (ISPs)
- access ISPs in turn must be interconnected
- resulting network of networks is very complex



At “center”: small # of well-connected large networks

- “tier-1” commercial ISPs (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- content provider networks (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

Packet delay: four sources

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

d_{proc} : nodal processing

- check bit errors
- determine output link: typically < microsecs
- d_{queue} : queuing delay
- time waiting at output link for transmission
- depends on congestion level of router

d_{trans} : transmission delay:

- L : packet length (bits)
- R : link transmission rate (bps)
- $d_{\text{trans}} = L/R$

d_{prop} : propagation delay:

- d : length of physical link
- s : propagation speed ($\sim 2 \times 10^8$ m/sec)
- $d_{\text{prop}} = d/s$

throughput: rate (bits/time unit) at which bits are being sent from sender to receiver

- *instantaneous*: rate at given point in time
- *average*: rate over longer period of time

Layered Internet protocol stack

- *application*: supporting network applications: HTTP, IMAP, SMTP, DNS. Exchanges messages to implement some application service using *services* of transport layer
- *transport*: process-process data transfer: TCP, UDP. Transport-layer protocol transfers M (e.g., reliably) from one *process* to another, using services of network layer
- *network*: routing of datagrams from source to destination: IP, routing protocols. Network-layer protocol transfers transport-layer segment $[H_t | M]$ from one *host* to another, using link layer services
- *link*: data transfer between neighboring network elements: Ethernet, 802.11 (WiFi), PPP. Link-layer protocol transfers datagram $[H_n | H_t | M]$ from *host* to neighboring host, using network-layer services
- *physical*: bits “on the wire”

Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software
- no need to write software for network-core devices
- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation

Client-server paradigm

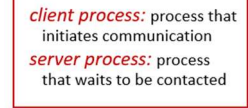
- server: always-on host, permanent IP address, often in data centers, for scaling
- clients:contact, communicate with server, may be intermittently connected, may have dynamic IP addresses, do *not* communicate directly with each other. examples: HTTP, IMAP, FTP

Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
- peers are intermittently connected and change IP addresses
- example: P2P file sharing [BitTorrent]

Processes communicating: program running within a host

- within same host, two processes communicate using inter-process communication (defined by OS)
- processes in different hosts communicate by exchanging messages



Sockets

- process sends/receives messages to/from its socket
- socket analogous to door
 - + sending process shoves message out door
 - + sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
- + two sockets involved: one on each side

Internet transport protocols services

Transmission Control Protocol TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won’t overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *connection-oriented*: setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security
- User Datagram Protocol UDP service:**
- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Web page

- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects*, each addressable by a *URL*, e.g.,

$$\underbrace{\text{www.someschool.edu}}_{\text{host name}} / \underbrace{\text{someDept/pic.gif}}_{\text{path name}}$$

HTTP: hypertext transfer protocol

- Web’s application-layer protocol
- client/server model:
 - + *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - + *server*: Web server sends (using HTTP protocol) objects in response to requests
- HTTP uses TCP:**
- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP connections: two types

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed
- Non-persistent HTTP**
- TCP connection opened
- at most one object sent over TCP connection
- TCP connection closed
- downloading multiple objects required multiple connections

Example: User enters URL:

- www.someSchool.edu/someDepartment/home.index
- 1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80
- 1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80 “accepts” connection, notifying client
- 2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index
- 3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket
- 4. HTTP server closes TCP connection.
- 5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects
- 6. Steps 1-5 repeated for each of 10 jpeg objects

RTT: time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time
- Non-persistent HTTP response time = 2RTT+ file transmission time*

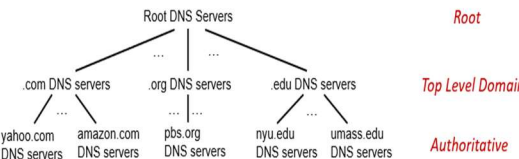
HTTP request messages

- **POST method**: web page often includes form input, user input sent from client to server in entity body of HTTP POST request message
- **HEAD method**: requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method
- **GET method** (for sending data to server): include user data in URL field of HTTP GET request message
- **PUT method**: uploads new file (object) to server, completely replaces file that exists at specified URL with content in entity body of POST HTTP request message
- HTTP response status codes**
- 200 OK: request succeeded, requested object later in this message
- 301 Moved Permanently: requested object moved, new location specified later in this message (in Location: field)
- 400 Bad Request: request msg not understood by server
- 404 Not Found: requested document not found on this server
- 505 HTTP Version Not Supported

DNS: Domain Name System

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names (address/name translation)
- DNS services:**
- hostname-to-IP-address translation
- host aliasing: canonical, alias names
- mail server aliasing
- load distribution

DNS: a distributed, hierarchical database



Client wants IP address for www.amazon.com:

- queries root server to find .com DNS server
- queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

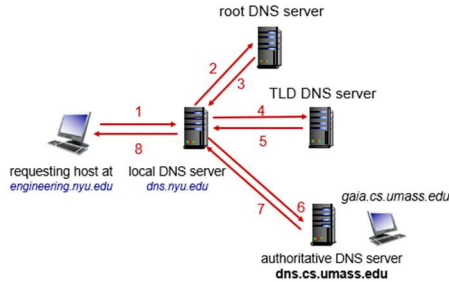
- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain
- Top-Level Domain (TLD) servers:**
- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk,
- Network Solutions: authoritative registry for .com, .net TLD

authoritative DNS servers:

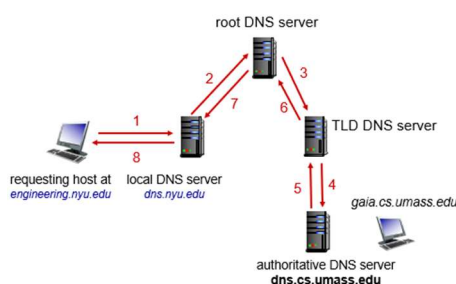
- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

DNS name resolution:

- **Iterated query:** contacted server replies with name of server to contact



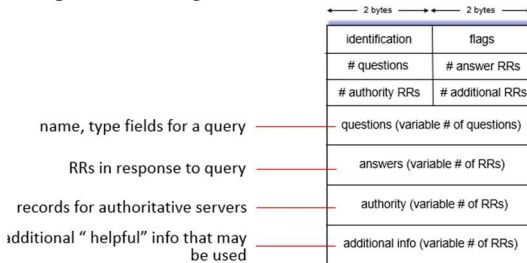
- **Recursive query:** puts burden of name resolution on contacted name server



DNS records

- DNS: distributed database storing resource records (RR)
- RR format: (name, value, type, ttl)
- type=A: name is hostname, value is IP address
- type=NS: name is domain (e.g., foo.com), value is hostname of authoritative name server for this domain
- type=CNAME: name is alias name for some "canonical" (the real) name, value is canonical name
- type=MX: value is name of SMTP mail server associated with name

DNS protocol messages



- identification: 16 bit # for query, reply to query uses same #
- flags: query or reply, recursion desired, recursion available
- reply is authoritative

- **multiplexing as sender:** handle data from multiple sockets, add transport header (later used for demultiplexing)
- **demultiplexing as receiver:** use header info to deliver received segments to correct socket

How demultiplexing works:

- host receives IP datagrams: each datagram has source IP address, destination IP address, each datagram carries one transport-layer segment, each segment has source, destination port number
- host uses IP addresses & port numbers to direct segment to appropriate socket

UDP: User Datagram Protocol

- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:lost, delivered out-of-order to app
- *connectionless*:no handshaking between UDP sender, receiver, each UDP segment handled independently of others
- Why is there a UDP**
- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control: UDP can blast away as fast as desired!, can function in the face of congestion

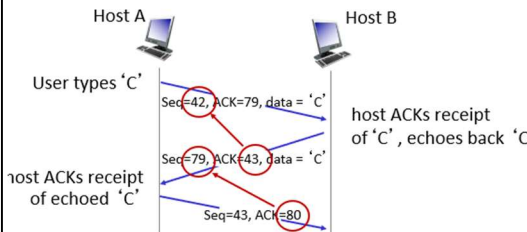
Internet checksum

- Goal:* detect errors (*i.e.*, flipped bits) in transmitted segment
- Sender:
 - + treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
 - + checksum: addition (one's complement sum) of segment content
 - + checksum value put into UDP checksum field
- Receiver:
 - + compute checksum of received segment
 - + check if computed checksum equals checksum field value: not equal - error detected, equal - no error detected

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	0	1	1	1	0	1	1	0	1	1	0	1	1	0	1
sum	1	0	1	1	1	0	1	1	0	1	1	1	0	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

TCP segment structure

- Sequence numbers:* byte stream "number" of first byte in segment's data
- Acknowledgements:* seq # of next byte expected from other side, cumulative ACK



TCP round trip time, timeout

- **Time out:** The time it takes for a packet to go from sender to receiver and for the acknowledgment (ACK) to return
- **Why do we need time out:** Because packets may be lost in the network. If the sender does not receive an ACK within the timeout period, it retransmits the packet to ensure reliable communication.
- **How to set TCP timeout value?** Longer than RTT, but RTT varies! *too short:* premature timeout, unnecessary retransmissions; *too long:* slow reaction to segment loss
- **How to estimate RTT?**
- SampleRTT:measured time from segment transmission until ACK receipt
- SampleRTT will vary, want estimated RTT "smoother"

Principles of congestion control

- Congestion: "too many sources sending too much data too fast for *network* to handle"
- manifestations: long delays (queueing in router buffers), packet loss (buffer overflow at routers)

TCP congestion control: AIMD

- approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

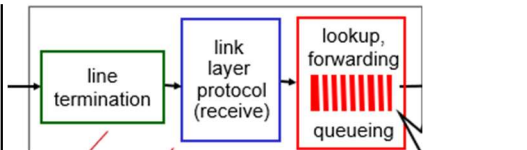
Two key network-layer functions

- *forwarding:* move packets from a router's input link to appropriate router output link
- *routing:* determine route taken by packets from source to destination

Data plane:

- *local,* per-router function
- determines how datagram arriving on router input port is forwarded to router output port
- Control plane**
- *network-wide* logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
 - + *traditional routing algorithms:* implemented in routers
 - + *software-defined networking (SDN):* implemented in (remote) servers

Input port functions



decentralized switching:

- using header field values, lookup output port using forwarding table in input port memory ("*match plus action*")
- goal: complete input port processing at 'line speed'
- **input port queueing:** if datagrams arrive faster than forwarding rate into switch fabric
- destination-based forwarding: forward based only on destination IP address (traditional)
- generalized forwarding: forward based on any set of header field values

- Longest prefix matching:** when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Switching fabrics

- transfer packet from input link to appropriate output link
- switching rate: rate at which packets can be transfer from inputs to outputs
- 3 major types of switching fabrics:memory, bus, interconnection network

Switching via memory

- traditional computers with switching under direct control of CPU
- packet copied to system's memory
- speed limited by memory bandwidth

Switching via a bus

- datagram from input port memory to output port memory via a shared bus
- *bus contention:* switching speed limited by bus bandwidth

Switching via interconnection network:

- Crossbar, Clos networks, other interconnection nets
- initially developed to connect processors in multiprocessor

Input port queueing:

- Occurs If switch fabric slower than input ports combined
- queueing delay and loss due to input buffer overflow!

Output port queueing:

- *Buffering* required when datagrams arrive from fabric faster than link transmission rate.
- *Scheduling discipline* chooses among queued datagrams for transmission
- buffering when arrival rate via switch exceeds output line speed
- *queueing (delay) and loss due to output port buffer overflow!*

- packet scheduling:** deciding which packet to send next on link

- **FCFS:** packets transmitted in order of arrival to output port
- **Priority scheduling:** arriving traffic classified, queued by class, send packet from highest priority queue that has buffered packets
- *Round Robin (RR) scheduling:* arriving traffic classified, queued by class, server cyclically, repeatedly scans class queues, sending one complete packet from each class (if available) in turn
- *Weighted Fair Queueing:* generalized Round Robin, each class, *i*, has weight, w_i , and gets weighted amount of service in each cycle: $\frac{w_i}{\sum_j w_j}$

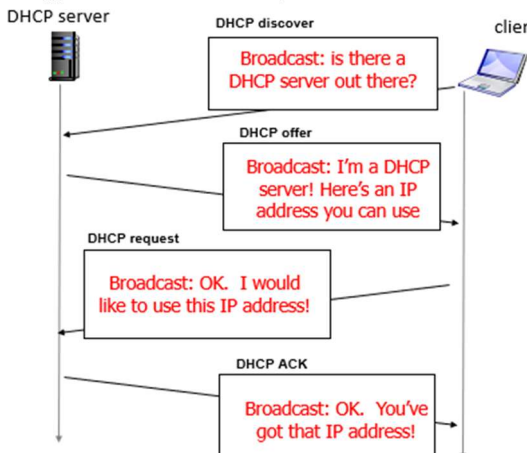
- **IP address:** 32-bit identifier associated with each host or router *interface*
- **interface:** connection between host/router and physical link
- **Subnet:** device interfaces that can physically reach each other without passing through an intervening router
- **IP addresses have structure:**

- + subnet part: devices in same subnet have common high order bits
- + host part: remaining low order bits
- **Recipe for defining subnets:**
- + detach each interface from its host or router, creating "islands" of isolated networks
- + each isolated network is called a *subnet*

DHCP: Dynamic Host Configuration Protocol

- goal: host *dynamically* obtains IP address from network server when it "joins" network
- + can renew its lease on address in use

- + allows reuse of addresses (only hold address while connected/on)
- + support for mobile users who join/leave network



network address translation NAT: all devices in local network share just one IPv4 address as far as outside world is concerned

IPv6 datagram format

- priority: identify priority among datagrams in flow
- flow label: identify datagrams in same "flow." (concept of "flow" not well defined).
- What's missing (compared with IPv4): no checksum (to speed processing at routers), no fragmentation/reassembly, no options

Interfaces communicating

sending side:

- encapsulates datagram in frame

receiving side:

- looks for errors, reliable data transfer, flow control, etc.
- extracts datagram, passes to upper layer at receiving side

Error detection

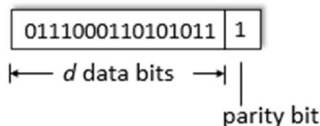
EDC: error detection and correction bits (e.g., redundancy)

D: data protected by error checking, may include header fields

Parity checking

single bit parity: detect single bit errors

two-dimensional parity: detect *and correct* single bit errors



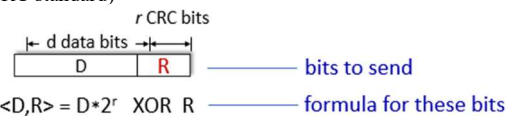
Even/odd parity: set parity bit so there is an even/odd number of 1's

At receiver:

- compute parity of d received bits
- compare with received parity bit – if different than error detected

Cyclic Redundancy Check (CRC)

- more powerful error-detection coding
- D: data bits (given, think of these as a binary number)
- G: bit pattern (generator), of $r+1$ bits (given, specified in CRC standard)

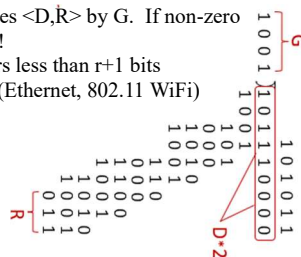


sender: compute r CRC bits, R , such that $\langle D, R \rangle$ exactly divisible by $G \pmod{2}$

- receiver knows G , divides $\langle D, R \rangle$ by G . If non-zero remainder: error detected!

- can detect all burst errors less than $r+1$ bits

- widely used in practice (Ethernet, 802.11 WiFi)



each interface on LAN

- has unique 48-bit MAC address
- has a locally unique 32-bit IP address

32-bit IP address:

- network-layer address for interface

- used for layer 3 (network layer) forwarding

- e.g.: 128.119.40.136

MAC (or LAN or physical or Ethernet) address:

- function: used "locally" to get frame from one interface to another physically-connected interface (same subnet, in IP-addressing sense)

- 48-bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable

- e.g.: 1A-2F-BB-76-09-AD

ARP: address resolution protocol: is a network protocol used to find the MAC address of a device when you already know its IP address on a local network.

Why it's needed: Devices communicate locally using **MAC addresses**, not IP addresses. When your computer wants to send data to another device on the same LAN, it uses ARP to ask: "Who has this IP address? Tell me your MAC address."

ARP table: each IP node (host, router) on LAN has table

- IP/MAC address mappings for some LAN nodes:

< IP address; MAC address; TTL >

- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

ARP protocol in action

1. A broadcasts ARP query, containing B's IP addr
- destination MAC address = FF-FF-FF-FF-FF-FF
- all nodes on LAN receive ARP query
2. B replies to A with ARP response, giving its MAC address
3. A receives B's reply, adds B entry into its local ARP table

Ethernet frame structure



- addresses: 6 byte source, destination MAC addresses

+ if adapter receives frame with matching destination address, or with broadcast address (e.g., ARP packet), it passes data in frame to network layer protocol

+ otherwise, adapter discards frame

- type: indicates higher layer protocol

+ mostly IP but others possible, e.g., Novell IPX, AppleTalk

+ used to demultiplex up at receiver

- CRC: cyclic redundancy check at receiver

+ error detected: frame is dropped

Ethernet switch

- Switch is a link-layer device: takes an *active* role

+ store, forward Ethernet (or other type of) frames

+ examine incoming frame's MAC address, *selectively* forward frame to one-or-more outgoing links when frame is to be forwarded on segment, uses CSMA/CD to access segment

- transparent: hosts *unaware* of presence of switches

- plug-and-play, self-learning: switches do not need to be configured

Switch: multiple simultaneous transmissions

- hosts have dedicated, direct connection to switch

- switches buffer packets

- Ethernet protocol used on *each* incoming link, so:

+ no collisions; full duplex

+ each link is its own collision domain

- switching: A-to-A' and B-to-B' can transmit simultaneously, without collisions. But A-to-A' and C to A' can *not* happen simultaneously

Wireless link characteristics:

1. multipath

multipath propagation: radio signal reflects off objects ground, built environment, arriving at destination at slightly different times

Coherence time:

- amount of time bit is present in channel to be received

- influences maximum possible transmission rate, since coherence times can not overlap

- inversely proportional to frequency, receiver velocity

2. fading

Wireless radio signal attenuates (loses power) as it propagates (free space "path loss")

3. noise

- interference from other sources on wireless network frequencies: motors, appliances

- SNR: signal-to-noise ratio: larger SNR – easier to extract signal from noise (a "good thing")

- SNR versus BER tradeoff

+ *given physical layer:* increase power -> increase SNR -> decrease BER

+ SNR may change with mobility: dynamically adapt

physical layer (modulation technique, rate)

4. hidden terminals

B, A hear each other

B, C hear each other

A, C can not hear each other means A, C unaware of their interference at B

Code Division Multiple Access (CDMA)

- unique "code" assigned to each user; i.e., code set partitioning

+ all users share same frequency, but each user has own

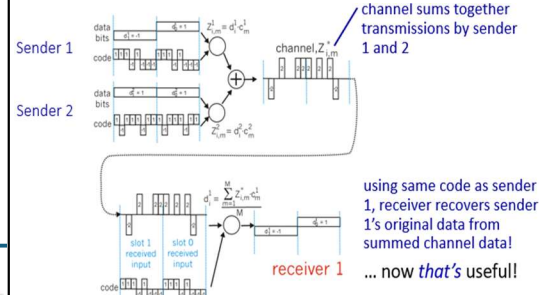
"chipping" sequence (i.e., code) to encode data

+ allows multiple users to "coexist" and transmit simultaneously with minimal interference (if codes are "orthogonal")

- **encoding:** inner product: (original data) X (chipping sequence)

- **decoding:** summed inner-product: (encoded data) X (chipping sequence)

CDMA: two-sender interference



IEEE 802.11: multiple access

- avoid collisions: 2^+ nodes transmitting at same time

- 802.11: CSMA - sense before transmitting

+ don't collide with detected ongoing transmission by another node

- 802.11: *no* collision detection!

+ difficult to sense collisions: high transmitting signal, weak received signal due to fading

+ can't sense all collisions in any case: hidden terminal, fading

+ goal: *avoid collisions*: CSMA/Collision Avoidance

Avoiding collisions: sender "reserves" channel use for data frames using small reservation packets

- sender first transmits *small* request-to-send (RTS) packet to BS using **CSMA**. RTSs may still collide with each other (but they're short)

- BS broadcasts clear-to-send CTS in response to RTS

- CTS heard by all nodes

+ sender transmits data frame

+ other stations defer transmissions

Elements of 4G LTE architecture

1. Base station:

- at "edge" of carrier's network

- manages wireless radio resources, mobile devices in its coverage area ("cell")

- coordinates device authentication with other elements

- similar to WiFi AP but:

+ active role in user mobility

+ coordinates with nearby base stations to optimize radio use

- LTE jargon: eNode-B

2. Home Subscriber Service

- stores info about mobile devices for which the HSS's network is their "home network"

- works with MME in device authentication

3. Serving Gateway (S-GW), PDN Gateway (P-GW)

- lie on data path from mobile to/from Internet

- P-GW

+ gateway to mobile cellular network

+ Looks like any other internet gateway router

+ provides NAT services

- other routers: extensive use of tunneling

4. Mobility Management Entity

- device authentication (device-to-network, network-to-device) coordinated with mobile home network HSS

- mobile device management:

+ device handover between cells

+ tracking/paging device location

- path (tunneling) setup from mobile device to P-GW

Personal area networks: Bluetooth

- less than 10 m diameter

- replacement for cables (mouse, keyboard, headphones)

- ad hoc: no infrastructure
- 2.4-2.5 GHz ISM radio band, up to 3 Mbps
- master controller / client devices: master polls clients, grants requests for client transmissions
- TDM, 625 msec sec. slot
- FDM: sender uses 79 frequency channels in known, pseudo-random order slot-to-slot (spread spectrum)
- other devices/equipment not in piconet only interfere in some slots

- **parked mode:** clients can “go to sleep” (park) and later wakeup (to preserve battery)
- **bootstrapping:** nodes self-assemble (plug and play) into piconet

Mobility with indirect routing: comments

- triangle routing: inefficient when correspondent and mobile are in same network
- mobile moves among visited networks: transparent to correspondent!

+ registers in new visited network

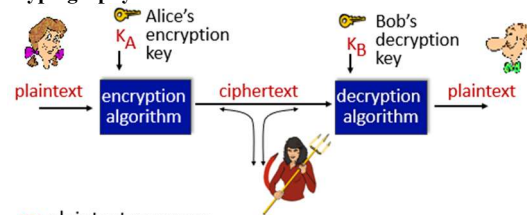
+ new visited network registers with home HSS

+ datagrams continue to be forwarded from home network to mobile in new network

What is network security?

- confidentiality: only sender, intended receiver should “understand” message contents: sender encrypts message, receiver decrypts message
- authentication: sender, receiver want to confirm identity of each other
- message integrity: sender, receiver want to ensure message not altered (in transit, or afterwards) without detection
- access and availability: services must be accessible and available to users

cryptography



m: plaintext message

$K_A(m)$: ciphertext, encrypted with key K_A

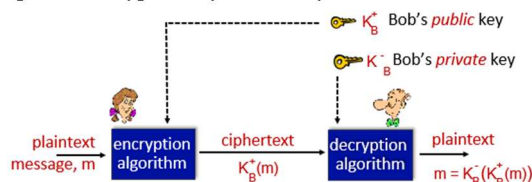
$m = K_B(K_A(m))$

symmetric key crypto: requires sender, receiver know shared secret key

e.g., key is knowing substitution pattern in mono alphabetic substitution cipher

public key crypto:

- radically different approach [Diffie-Hellman76, RSA78]
- sender, receiver do *not* share secret key
- **public encryption** key known to *all*
- **private decryption** key known only to receiver



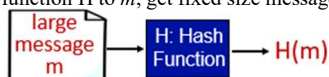
Public key encryption algorithms requirements:

- 1 need $K_B^+(\cdot)$ and $K_B^-(\cdot)$ such that $K_B^-(K_B^+(m)) = m$
- 2 given public key K_B^+ , it should be impossible to compute private key K_B^-

RSA: Rivest, Shamir, Adelson algorithm

Message digests:

- computationally expensive to public-key-encrypt long messages
- goal: fixed-length, easy- to-compute digital “fingerprint”
- apply hash function H to m , get fixed size message digest, $H(m)$



Hash function properties:

- many-to-1
- produces fixed-size msg digest (fingerprint)

- given message digest x , computationally infeasible to find m such that $x = H(m)$

Hash function algorithms

- MD5 hash function widely used (RFC 1321)
- + computes 128-bit message digest in 4-step process.
- + arbitrary 128-bit string x , appears difficult to construct msg m whose MD5 hash is equal to x
- SHA-1 is also used
- + US standard [NIST, FIPS PUB 180-1]
- + 160-bit message digest

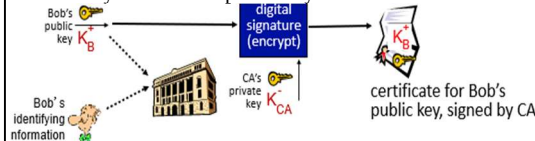
Hash function algorithms

2 types of Hash

- MD5 hash function widely used (RFC 1321)
- + computes 128-bit message digest in 4-step process.
- + arbitrary 128-bit string x , appears difficult to construct msg m whose MD5 hash is equal to x
- SHA-1 is also used
- + US standard [NIST, FIPS PUB 180-1]
- + 160-bit message digest

Public key Certification Authorities (CA)

- certification authority (CA): binds public key to particular entity, E
- entity (person, website, router) registers its public key with CE provides “proof of identity” to CA
- + CA creates certificate binding identity E to E’s public key
- + certificate containing E’s public key digitally signed by CA: CA says “this is E’s public key”



- when Alice wants Bob’s public key:

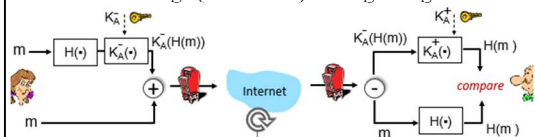
+ gets Bob’s certificate (Bob or elsewhere)

+ apply CA’s public key to Bob’s certificate, get Bob’s public key

Secure e-mail: integrity, authentication

Alice wants to send m to Bob, with *message integrity*, *authentication*

- Alice digitally signs hash of her message with her private key, providing integrity and authentication
- sends both message (in the clear) and digital signature



Alice uses three keys: her private key, Bob’s public key, new symmetric key

t-tls: cryptographic keys

- considered bad to use same key for more than one cryptographic function
- + different keys for message authentication code (MAC) and encryption

- 4 keys:

K_c : encryption key for data sent from client to server

M_c : MAC key for data sent from client to server

K_s : encryption key for data sent from server to client

M_s : MAC key for data sent from server to client

- keys derived from key derivation function (KDF)

+ takes master secret and (possibly) some additional random data to create new keys